Caderno de Exercícios

Curso de Grafana — Visualizando dados

Instrutor: Fabio Silva

Carga horária: 16h (4 noites)

18:30 às 22:30, com 20 min de intervalo

> Este caderno contém os exercícios guiados pelo instrutor em cada noite.
> Imprima ou abra em PDF lado a lado com o terminal.

Caderno de Exercícios — Curso de Grafana

Instrutor: Fabio Silva

Carga horária total: 16h (4 dias × 4h)

Duração de cada aula: 18:30 às 22:30, com 20 min de intervalo

Este caderno acompanha o curso **Grafana: Visualizando dados**. Ele contém todos os exercícios que o aluno deve executar em cada dia, na ordem em que o professor guia a turma. As respostas esperadas (gabarito) estão na seção final.

Aviso: todos os laboratórios usam o usuário `admin / admin` apenas no contexto da sala de aula. Em qualquer ambiente real, troque a senha no primeiro acesso.

Índice

- [Dia 1 — Fundamentos e navegação](#dia-1--fundamentos-e-navegação)
- [Dia 2 — Visualizações, variáveis e alertas](#dia-2--visualizações-variáveis-e-alertas)
- [Dia 3 — Instalação no Ubuntu (prática guiada)](#dia-3--instalação-no-ubuntu-prática-guiada)
- [Dia 4 — Temas, provisionamento, multi-stack e encerramento](#dia-4--temas-provisionamento-multi-stack-e-encerramento)
- [Gabarito](#gabarito)

Dia 1 — Fundamentos e navegação

Objetivo do dia: entender observabilidade, subir o ambiente Docker, navegar no Grafana com dashboards pré-configurados e praticar a criação do primeiro painel do zero.

Roteiro da noite (4h)

Horário	Bloco	Material
18:30 – 19:20	Teoria: observabilidade e Grafana	Slides 1–20
19:20 – 19:40	Intervalo (20 min)	—
19:40 – 20:30	Demonstração ao vivo: docker compose up	Slides 21–30, lab/dia1/
20:30 – 22:30	Exercícios práticos (aluno)	Exercícios 1.1 a 1.7

18:30 – 19:20	Teoria: observabilidade e Grafana	Slides 1–20
19:20 – 19:40	Intervalo (20 min)	—

19:40 – 20:30	Demonstração ao vivo: docker compose up	Slides 21–30, lab/dia1/
20:30 – 22:30	Exercícios práticos (aluno)	Exercícios 1.1 a 1.7

18:30 – 19:20	Teoria: observabilidade e Grafana	Slides 1–20
19:20 – 19:40	Intervalo (20 min)	—
19:40 – 20:30	Demonstração ao vivo: docker compose up	Slides 21–30, lab/dia1/
20:30 – 22:30	Exercícios práticos (aluno)	Exercícios 1.1 a 1.7

19:20 – 19:40	Intervalo (20 min)	—
19:40 – 20:30	Demonstração ao vivo: docker compose up	Slides 21–30, lab/dia1/
20:30 – 22:30	Exercícios práticos (aluno)	Exercícios 1.1 a 1.7

19:40 – 20:30	Demonstração ao vivo: docker compose up	Slides 21–30, lab/dia1/
20:30 – 22:30	Exercícios práticos (aluno)	Exercícios 1.1 a 1.7

| 20:30 – 22:30 | Exercícios práticos (aluno) | Exercícios 1.1 a 1.7 |

Pré-aula (verificar antes de começar)

- [ ] Docker Desktop instalado e em execução
- [ ] 4 GB de RAM livre para os containers
- [ ] Portas 3000, 9090 e 9100 livres

Exercício 1.1 — Subir o ambiente

- Abra um terminal e vá para a pasta do lab do Dia 1:

```bash
cd lab/dia1
ls
```

Você deve ver: `docker-compose.yml`, `prometheus.yml`, `alerts.yml` e a pasta `grafana/`.

- Suba a stack:

```bash
docker compose up -d
```

- Confira o status:

```bash
docker compose ps
```

- Aguarde ~30s e abra o navegador em **http://localhost:3000** (login `admin` / `admin`).
- Mude a senha quando o Grafana pedir.

Tempo esperado: 3 min. Se algum container ficar em `Restarting`, peça ajuda ao instrutor.

Exercício 1.2 — Confirmar o datasource

- No Grafana, vá em **Connections** → **Data sources** (engrenagem lateral).
- Você deve ver **Prometheus** listado e com um ícone verde.
- Clique em **Prometheus** → **Save & test** (botão inferior direito).
- O resultado deve ser: `datasource is working. 1 data source found`.

Exercício 1.3 — Explorar o dashboard provisionado

- No menu lateral, clique em **Dashboards** → **Laboratório**.
- Abra o dashboard **01 - Visão geral do host**.
- Ajuste o intervalo de tempo para os últimos **5 minutos** (canto superior direito).
- Identifique cada painel e responda:

- Qual painel mostra o uso de **CPU**?

- Qual painel mostra **memória disponível**?

- Qual painel tem o tipo **stat**? E **time series**?

- Clique no título do painel **CPU por core (5m)** e em **Edit**.
- Em **Panel options**, troque o título para `CPU por core - meu host`.
- Clique em **Apply** e depois em **Save dashboard** (ícone de disquete, no topo).

Exercício 1.4 — Explorar a UI

- Identifique na barra lateral esquerda os ícones:

- **Search** (lupa)

- **Dashboards** (quatro quadradinhos)

- **Explore** (bússola)

- **Alerting** (sino)

- Clique em **Explore**.
- Selecione o datasource **Prometheus**.
- No campo de query, digite exatamente:

```promql
node_cpu_seconds_total
```

- Clique em **Run query** (canto superior direito).
- Observe a tabela de labels. Mude para a visualização **Time series** (botão à direita do título do painel).
- Agora experimente:

```promql
rate(node_cpu_seconds_total{mode="idle"}[1m])
```

O que mudou?

Exercício 1.5 — Criar o primeiro dashboard

- Volte para **Dashboards** → **New** → **New dashboard**.
- Clique em **Add visualization**.
- O datasource **Prometheus** já deve estar selecionado.
- Cole a query:

```promql
100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100)
```

- Mude o **Panel title** para `Uso de CPU do host (%)`.
- Em **Standard options** → **Unit**, escolha `Percent (0-100)`.
- Em **Thresholds**, ajuste: verde até 60, amarelo até 80, vermelho acima.
- Clique em **Apply**.
- Clique em **Save dashboard** → nome: `Meu primeiro dashboard`.

Exercício 1.6 — Adicionar mais um painel

- No dashboard `Meu primeiro dashboard`, clique em **Add** → **Visualization**.
- Visualização: **Time series**.
- Query:

```promql
node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes
```

- Título: `Memória em uso (bytes)`.
- Em **Unit**, escolha `bytes`.
- Em **Graph styles**, marque **Fill opacity = 20**.
- Clique em **Apply** e salve.

Exercício 1.7 — Encerrar o ambiente

- No terminal:

```bash
docker compose down
```

- Confirme que os containers pararam:

```bash
```

docker compose ps

```

A saída deve estar vazia.

**Importante:** deixamos os volumes intactos (`-v` não foi usado). No Dia 2 vamos usar outra stack; o Grafana do Dia 1 não conflita porque muda o nome do projeto (`-p`). **Não dê `down -v` no Dia 1**, a menos que o instrutor peça.

---

## Dia 2 — Visualizações, variáveis e alertas

**Objetivo do dia:** dominar todos os tipos de visualização mais usados, criar dashboards dinâmicos com variáveis e configurar alertas.

### Roteiro da noite (4h)

| Horário | Bloco | Material |
|---|---|---|
| 18:30 – 19:20 | Teoria: tipos de painel, variáveis, alertas | Slides 1–22 do Dia 2 |
| 19:20 – 19:40 | Intervalo | — |
| 19:40 – 20:30 | Demonstração ao vivo | lab/dia2/ |
| 20:30 – 22:30 | Exercícios práticos (aluno) | Exercícios 2.1 a 2.7 |
| 18:30 – 19:20 | Teoria: tipos de painel, variáveis, alertas | Slides 1–22 do Dia 2 |
| 19:20 – 19:40 | Intervalo | — |
| 19:40 – 20:30 | Demonstração ao vivo | lab/dia2/ |
| 20:30 – 22:30 | Exercícios práticos (aluno) | Exercícios 2.1 a 2.7 |
| 18:30 – 19:20 | Teoria: tipos de painel, variáveis, alertas | Slides 1–22 do Dia 2 |
| 19:20 – 19:40 | Intervalo | — |
| 19:40 – 20:30 | Demonstração ao vivo | lab/dia2/ |
| 20:30 – 22:30 | Exercícios práticos (aluno) | Exercícios 2.1 a 2.7 |
| 19:20 – 19:40 | Intervalo | — |
| 19:40 – 20:30 | Demonstração ao vivo | lab/dia2/ |
| 20:30 – 22:30 | Exercícios práticos (aluno) | Exercícios 2.1 a 2.7 |
| 19:40 – 20:30 | Demonstração ao vivo | lab/dia2/ |
| 20:30 – 22:30 | Exercícios práticos (aluno) | Exercícios 2.1 a 2.7 |

| 20:30 – 22:30 | Exercícios práticos (aluno) | Exercícios 2.1 a 2.7 |
```

Pré-aula

- [ ] Diretório `lab/dia2/` pronto (fornecido pelo instrutor).

Exercício 2.1 — Subir a stack do Dia 2

- Em outro terminal (ou após parar a do Dia 1):

```bash
cd lab/dia2
docker compose up -d
```

- Confira os containers: `grafana`, `prometheus`, `node-exporter`, `cadvisor`, `alertmanager`.
- Abra **http://localhost:3000**.

Exercício 2.2 — Usar variáveis

- Abra o dashboard **02 - Host dinâmico (variáveis)**.
- No topo, localize o dropdown **host**. Selecione apenas um host.
- Observe todos os painéis atualizarem.
- Selecione **All** e veja que aparece um item por host em cada legenda.
- Adicione uma **segunda variável** do tipo `Custom` chamada `cpu_mode` com os valores `idle,user,system,iowait`.
- Em uma das queries existentes, troque `mode="idle"` por `mode=~"$cpu_mode"`.
- Salve o dashboard como `02 - Host dinâmico (variáveis) — meu`.

Exercício 2.3 — Explorar o painel Gauge

- No mesmo dashboard, clique em **Edit** no painel **Memória ($host)**.
- Observe o tipo: **gauge**.
- Mude o **Unit** para `Percent (0-100)`.
- Em **Thresholds**, defina 0–70 verde, 70–85 amarelo, 85+ vermelho.
- Em **Gauge** → **Show threshold markers**, marque como ativo.
- Clique em **Apply** e salve.

Exercício 2.4 — Explorar o painel Heatmap

- No painel **Heatmap de uso de CPU por hora do dia**, clique em **Edit**.
- O tipo é **heatmap**. Note que a query está com `format: heatmap`.
- Aumente o intervalo de tempo para **6 horas** (canto superior direito) e veja o mapa de cor.
- Em **Y-axis**, troque `linear` para `log` e veja a diferença visual.

Exercício 2.5 — Container dashboard (cAdvisor)

- Abra o dashboard **03 - Containers (cAdvisor)**.
- Identifique:

- Quais containers estão em uso no host.

- Qual container tem o maior consumo de CPU.

• Crie um novo painel **Time series** com a query:

```promql
rate(container_network_receive_bytes_total{name!=""}[5m])
```

• Título: `Rede recebida por container`.
• Salve.

Exercício 2.6 — Configurar alerta Prometheus

• Abra **http://localhost:9090/alerts**.
• Veja que já existem 3 regras: `HostCPUAlta`, `HostDiscoAlto`, `HostDown`.
• Acesse **http://localhost:9090/graph** e rode:

```promql
100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
```

• Para forçar um alerta, em outro terminal:

```bash
docker exec -it lab-node-exporter sh -c \
'dd if=/dev/urandom of=/dev/null bs=1M count=2000 &'
```

Espere 5 minutos e veja o alerta mudar para **FIRING** no Prometheus.

• Mate o processo:

```bash
docker exec -it lab-node-exporter pkill dd
```

Exercício 2.7 — Configurar alerta no Grafana

• No Grafana, vá em **Alerting** → **Alert rules** → **New alert rule**.
• Query:

```promql
100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
```

• Condition: `IS ABOVE 80`.
• **Folder**: `Laboratório`.
• **Evaluation group**: `1m` (evalua a cada 1 min), `for: 5m`.
• Clique em **Save and exit**.

• O alerta vai aparecer em **Alerting** → **Alert rules**. Após 5 min com CPU > 80%, ele fica em `Firing`.

Dia 3 — Instalação no Ubuntu (prática guiada)

Objetivo do dia: o aluno instala, do zero, Grafana + Prometheus + Node Exporter em uma máquina Ubuntu, usando os scripts comentados.

Roteiro da noite (4h)

Horário	Bloco	Material
18:30 – 19:20	Teoria: observabilidade e Grafana	Slides 1–20
19:20 – 19:40	Intervalo (20 min)	—
19:40 – 20:30	Demonstração ao vivo: docker compose up	Slides 21–30, lab/dia1/
20:30 – 22:30	Exercícios práticos (aluno)	Exercícios 1.1 a 1.7

18:30 – 19:20	Teoria: observabilidade e Grafana	Slides 1–20
19:20 – 19:40	Intervalo (20 min)	—
19:40 – 20:30	Demonstração ao vivo: docker compose up	Slides 21–30, lab/dia1/
20:30 – 22:30	Exercícios práticos (aluno)	Exercícios 1.1 a 1.7

18:30 – 19:20	Teoria: instalação, systemd, segurança	Slides 1–18 do Dia 3
19:20 – 19:40	Intervalo	—
19:40 – 22:30	Execução sequencial dos 6 scripts	lab/dia3/

19:20 – 19:40	Intervalo	—
19:40 – 22:30	Execução sequencial dos 6 scripts	lab/dia3/

| 19:40 – 22:30 | Execução sequencial dos 6 scripts | lab/dia3/ |

O instrutor guia passo a passo. O aluno roda comando por comando **com a tela projetada**.

Pré-aula

- [ ] VM ou máquina Ubuntu 22.04 LTS (≥ 2 vCPU, ≥ 4 GB RAM)
- [ ] Acesso a internet
- [ ] Usuário com `sudo`

Exercício 3.1 — Preparar o sistema

```
sudo bash 01-prepare.sh
```

Responda no caderno:

- Qual é o nome do usuário criado para o Grafana?
- O que faz `apt-transport-https`?

Exercício 3.2 — Instalar o Grafana

```
sudo bash 02-install-grafana.sh
```

Responda:

- Onde fica a chave GPG que foi baixada?
- Por que fixamos a versão (`11.3.0`)?

Exercício 3.3 — Configurar o Grafana

```
sudo bash 03-configure.sh
```

Responda:

- Qual a porta em que o Grafana passou a escutar?
- Onde o Grafana grava o banco SQLite?
- Qual é a senha padrão? Por que ela deve ser trocada?

Exercício 3.4 — Instalar o Prometheus

```
sudo bash 04-install-prometheus.sh
```

Responda:

- Por que criamos um usuário `prometheus` separado?
- Em qual diretório fica o `prometheus.yml`?
- O que faz a flag `--web.enable-lifecycle`?

Exercício 3.5 — Instalar o Node Exporter

```
sudo bash 05-install-node-exporter.sh
```

Responda:

- Em qual porta o Node Exporter responde?
- O Node Exporter guarda estado? Justifique.

Exercício 3.6 — Validar tudo

```
sudo bash 06-validate.sh
```

O script deve imprimir `[OK]` em todas as checagens. Se algum item estiver em `[FAIL]`, peça ajuda.

Exercício 3.7 — Customizar

• Altere a porta do Grafana para `3500` em `/etc/grafana/grafana.ini` e reinicie:

```bash
sudo sed -i 's/^http_port = 3000/http_port = 3500/' /etc/grafana/grafana.ini
sudo systemctl restart grafana-server
```

• Acesse **http://localhost:3500** e confirme.
• Restaure para `3000` no final.

Dia 4 — Temas, provisionamento, multi-stack e encerramento

Objetivo do dia: customização visual, provisionamento de dashboards por arquivo, multi-stack e revisão geral.

Roteiro da noite (4h)

Horário	Bloco	Material
18:30 – 19:20	Teoria: tipos de painel, variáveis, alertas	Slides 1–22 do Dia 2
19:20 – 19:40	Intervalo	—
19:40 – 20:30	Demonstração ao vivo	lab/dia2/
20:30 – 22:30	Exercícios práticos (aluno)	Exercícios 2.1 a 2.7

18:30 – 19:20	Teoria: tipos de painel, variáveis, alertas	Slides 1–22 do Dia 2
19:20 – 19:40	Intervalo	—
19:40 – 20:30	Demonstração ao vivo	lab/dia2/
20:30 – 22:30	Exercícios práticos (aluno)	Exercícios 2.1 a 2.7

18:30 – 19:20	Teoria: theming, plugins, Grafana Cloud	Slides 1–18 do Dia 4
19:20 – 19:40	Intervalo	—
19:40 – 21:00	Provisionamento file-based (exercícios 4.1–4.3)	Slides 19–28
21:00 – 22:00	Multi-stack: Loki, Tempo, Mimir	Slides 29–35
22:00 – 22:30	Revisão e Q&A	—

19:20 – 19:40	Intervalo	—
19:40 – 20:30	Demonstração ao vivo	lab/dia2/

20:30 – 22:30	Exercícios práticos (aluno)	Exercícios 2.1 a 2.7

19:40 – 21:00	Provisionamento file-based (exercícios 4.1–4.3)	Slides 19–28
21:00 – 22:00	Multi-stack: Loki, Tempo, Mimir	Slides 29–35
22:00 – 22:30	Revisão e Q&A	—

21:00 – 22:00	Multi-stack: Loki, Tempo, Mimir	Slides 29–35
22:00 – 22:30	Revisão e Q&A	—

| 22:00 – 22:30 | Revisão e Q&A | — |

Exercício 4.1 — Instalar plugin de pizza

- Acesse o container do Grafana do Dia 2:

```bash
docker exec -it lab-grafana bash
grafana-cli plugins install grafana-piechart-panel
exit
```

- Reinicie o Grafana:

```bash
docker compose restart grafana
```

- Crie um novo painel — o tipo **Pie chart** deve aparecer.

Exercício 4.2 — Exportar um dashboard como JSON

- No dashboard `Meu primeiro dashboard` (Dia 1), clique em **Share** → **Export**.
- Marque **Export schema for sharing** e clique em **Save to file**.
- Salve como `meu-dashboard.json`.
- Mova o arquivo para `lab/dia2/grafana/dashboards/`.
- Recarregue a pasta:

```bash
curl -X POST http://admin:admin@localhost:3000/api/user/stars/dashboard/lookup -H 'Content-Type: application/json' -d '{}'
```

(alternativa: reinicie o Grafana).

- O novo dashboard deve aparecer em **Laboratório**.

Exercício 4.3 — Trocar o tema

- No Grafana, vá em **Preferences** (avatar, canto superior direito).
- Troque o tema para **Dark**.
- Em **Dashboards** → **New** → **Import**, cole a URL `https://grafana.com/grafana/dashboards/1860` (Node Exporter Full).
- Selecione o datasource **Prometheus** e clique em **Import**.

Exercício 4.4 — Observabilidade de Logs com Grafana Loki (Docker ou Linux Nativo)

Neste exercício você vai praticar a ingestão e consulta de logs via **LogQL**.

Opção A: Via Docker Compose (Recomendado)

Se você está no ambiente Docker:

```
cd lab/dia4
docker compose up -d loki promtail ecommerce-app
```

O Promtail começará imediatamente a ler os logs dos containers e do sistema operacional.

Opção B: Via Linha de Comando no Linux Nativo (Injetando Logs de Teste)

No seu terminal Ubuntu, você pode gerar logs diretamente no sistema para o Grafana visualizar:

```
# Injetar um log de informação
logger -t minha-app -p local0.info '{"app": "loja", "status": 200, "msg": "Cliente realizou login com sucesso"}'

# Injetar um log de erro crítico com simulação de TraceID
logger -t minha-app -p local0.err '{"app": "loja", "status": 500, "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736", "msg": "Falha de conexão com gateway de pagamento"}'
```

No Grafana:

- Vá em **Connections** → **Data sources** → **Add data source** e selecione **Loki**.
- URL: `http://localhost:3100` (ou `http://loki:3100` se estiver em container).
- Clique em **Save & test**.
- Vá em **Explore** (ícone da bússola) e selecione o datasource **Loki**.
- Execute as queries LogQL:

```logql
# Ver todos os logs da aplicação
{job="docker-containers"}

# Filtrar apenas erros
{job="docker-containers"} |= "ERROR"

# Contar volume de erros por minuto
sum(count_over_time({job="docker-containers"} |= "ERROR" [1m]))
```

• Ou importe diretamente o dashboard pronto: **Dashboards** → **New** → **Import** → **Upload file**: selecione `lab/dashboards-prontos-importacao/04-central-logs-loki.json`.

Exercício 4.5 — Observabilidade de Traces com Grafana Tempo e OpenTelemetry

Traces permitem visualizar o caminho de uma requisição fim a fim entre microsserviços, medindo a latência de cada etapa (spans).

1. Subindo o Tempo e a Aplicação Instrumentada

```
cd lab/dia4
docker compose up -d tempo ecommerce-app
```

2. Configurando o Data Source Tempo no Grafana

• No Grafana, vá em **Connections** → **Data sources** → **Add data source** e escolha **Tempo**.
• URL: `http://localhost:3200` (ou `http://tempo:3200`).
• Na seção **Trace to logs**, configure:
- Data source: **Loki**
- Tags: `service.name`

• Clique em **Save & test**.

3. Consultando Traces no Grafana Explore

• Abra o **Explore** e selecione o datasource **Tempo**.
• Na aba **Search**, selecione:
- Service Name: `ecommerce-shop`

• Clique em **Run query**.
• Uma lista de transações recentes aparecerá na tela. Clique em qualquer uma delas:
- Você verá o gráfico em cascata (Gantt) mostrando o span raiz HTTP e os spans filhos: autenticação, consulta SQL e chamada ao gateway de pagamento.
- Observe quanto tempo cada etapa demorou!

Exercício 4.6 — A Santíssima Trindade: Navegando de Logs para Traces (1-Clique)

Este é o recurso mais poderoso da observabilidade moderna:

• Abra o **Explore** no datasource **Loki**.
• Busque pelos erros de pagamento:

```logql
{job="docker-containers"} |= "ERROR"
```

• Expanda a linha do log de erro.

• Localize o campo `trace_id`. Note que há um botão azul com o ícone do Tempo ao lado dele!

• Clique no botão azul: o Grafana abre uma divisão de tela trazendo o **Trace completo daquela transação com erro**, permitindo que você descubra a causa raiz em segundos sem abrir nenhum outro sistema!

Gabarito

Gabarito Dia 1

• **1.2**: o teste deve mostrar `datasource is working`.

• **1.3**: tipos: `stat` para os 4 indicadores do topo, `time series` para CPU e memória.

• **1.4**: o segundo exemplo mostra a **taxa** de uso de CPU idle por segundo (em segundos por segundo).

Gabarito Dia 2

• **2.6**: depois de ~5 min com `dd` rodando, o alerta `HostCPUAlta` deve estar em verde-claro **FIRING**.

Gabarito Dia 3

• **3.1**:

1. `grafana`
2. Permite que o `apt` use o protocolo HTTPS para acessar o repositório.

• **3.2**:

1. `/etc/apt/keyrings/grafana.gpg`
2. Para garantir reprodutibilidade: todos os alunos sobem a mesma versão.

• **3.3**:

1. `3000`
2. `/var/lib/grafana/grafana.db`
3. `admin` — é o padrão amplamente conhecido e a primeira coisa que um atacante tenta.

• **3.4**:

1. Princípio do menor privilégio: o serviço não roda como `root`.
2. `/etc/prometheus/prometheus.yml`
3. Permite recarregar a config sem reiniciar o processo, via `POST /-/reload`.

• **3.5**:

1. `9100`
2. Não. Ele apenas lê `/proc` e `/sys` e expõe via HTTP; o estado fica no Prometheus.

Gabarito Dia 4

• **4.3**: o dashboard 1860 é o "Node Exporter Full" — traz ~50 painéis prontos cobrindo CPU por core, RAM, swap, I/O de disco, filas de rede e conexões TCP.

• **4.4**: no Loki, o LogQL usa sintaxe de stream `{job="docker-containers"}` seguida de filtros de texto `|= "ERROR"`. A agregação `count_over_time[1m]` transforma os logs em uma métrica de volume ao longo do tempo.

• **4.5**: o Tempo armazena traces usando spans do padrão OpenTelemetry. Cada span mede uma operação atômica de microsserviço com atributos (ex: query SQL, status HTTP, rota) e timestamp de início/fim.

• **4.6**: a correlação entre Logs e Traces ocorre graças à injeção do atributo `trace_id` padronizado no formato JSON dos logs. O Grafana lê esse campo via regex nos Data Sources (Derived Fields) e constrói o link direto para o Tempo.

Bom curso! Em caso de dúvida durante a aula, levante a mão ou chame o instrutor. Procure revisar o caderno antes da próxima aula.